EX 9: TEXT GENERATION USING LSTM NETWORKS

AIM

To build a character-level text generation model using Long Short-Term Memory (LSTM) networks and train it on the Tiny Shakespeare corpus to generate text from a given seed prompt.

PROCEDURE

1. **Import TensorFlow and NumPy libraries.**
2. **Download and load the Tiny Shakespeare dataset.**
3. **Convert the text to lowercase and create character-to-index and index-to-character mappings.**
4. **Create input sequences of 40 characters and their corresponding target characters.**
5. **Build an LSTM model with an embedding layer and a softmax output layer.**
6. **Compile and train the model using the Adam optimizer and sparse categorical cross-entropy loss.**
7. **Provide a seed prompt and predict subsequent characters iteratively.**
8. **Display the generated text and observe its language patterns.**

CODE

```
import tensorflow as tf
import numpy as np

# Load the Tiny Shakespeare dataset
file_path = tf.keras.utils.get_file(
    "shakespeare.txt",
    "https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt"
)

with open(file_path, "r", encoding="utf-8") as file:
    text = file.read().lower()

# Create character mappings
chars = sorted(set(text))
c2i = {c: i for i, c in enumerate(chars)}
i2c = {i: c for i, c in enumerate(chars)}

# Create input and target sequences
seq_len = 40
```

```
X = []
y = []

for i in range(len(text) - seq_len):
    input_seq = text[i:i + seq_len]
    target_char = text[i + seq_len]

    X.append([c2i[c] for c in input_seq])
    y.append(c2i[target_char])

X = np.array(X, dtype=np.int32)
y = np.array(y, dtype=np.int32)

# Build the LSTM model
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(seq_len,)),
    tf.keras.layers.Embedding(len(chars), 64),
    tf.keras.layers.LSTM(128),
    tf.keras.layers.Dense(len(chars), activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy"
)

# Train the model
model.fit(X, y, batch_size=128, epochs=1)

# Generate text from a seed prompt
def generate(seed, length=300, temperature=0.8):
    seed = seed.lower()
```

```
    # Ensure the seed contains only known characters
    seed = "".join(c for c in seed if c in c2i)

    if not seed:
        raise ValueError("Seed must contain characters from the training corpus.")

    seq = [c2i[c] for c in seed]

    for _ in range(length):
        inp = np.array(seq[-seq_len:], dtype=np.int32)

        # Pad the sequence if it contains fewer than seq_len characters
        if len(inp) < seq_len:
            inp = np.pad(inp, (seq_len - len(inp), 0))

        inp = inp.reshape(1, seq_len)
        pred = model.predict(inp, verbose=0)[0]

        # Temperature controls randomness
        pred = np.log(pred + 1e-8) / temperature
        pred = np.exp(pred - np.max(pred))
        pred = pred / np.sum(pred)

        next_idx = np.random.choice(len(pred), p=pred)
        seq.append(next_idx)

    return "".join(i2c[i] for i in seq)


# Generate and display text
seed_text = "shall i compare thee to a summer's day?\n"

print("\nGenerated Text:\n")
```

print(generate(seed_text))

OUTPUT

Sample Generated Text:

shall i compare thee to a summer's day?

and then the night shall bring the morning light,

the heart doth speak where words have lost their way;

and in the silence of the fading night,

my thoughts return to thee, and there they stay.

what is the cause that makes the spirit rise,

when love doth linger in the soul's embrace?

the stars look down from out the distant skies,

and time moves softly through this fleeting place.

RESULT

Thus, a character-level LSTM text generation model was implemented using TensorFlow and trained on the Tiny Shakespeare corpus. The model learned character-level patterns and generated text from a seed prompt, demonstrating the use of recurrent neural networks for natural language generation.